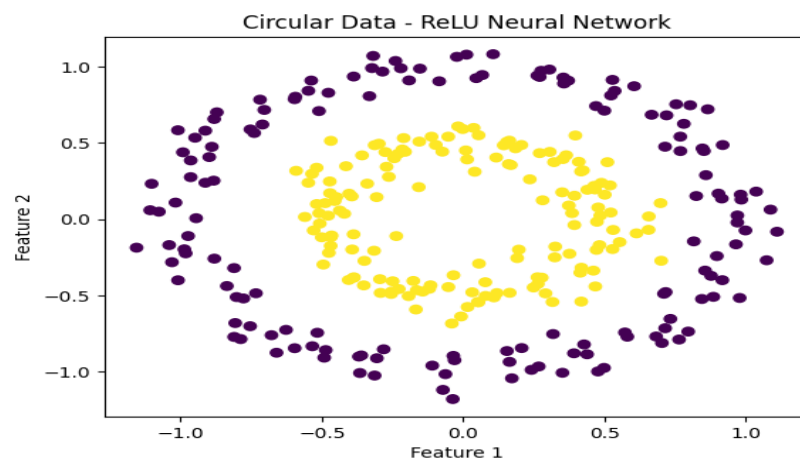


EXP 21:

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.neural_network import MLPClassifier
X, y = make_circles(n_samples=300, noise=0.08, factor=0.5,
                    random_state=42)
model = MLPClassifier(hidden_layer_sizes=(10, 10),
                      activation='relu',
                      max_iter=1000,
                      random_state=42)
model.fit(X, y)
y_pred = model.predict(X)
accuracy = model.score(X, y)
print("Activation Function: ReLU")
print("Input Samples:", len(X))
print("Accuracy:", round(accuracy * 100, 2), "%")
plt.scatter(X[:, 0], X[:, 1], c=y_pred)
plt.title("Circular Data - ReLU Neural Network")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT:



EXP 22:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neural_network import MLPClassifier

X, y = make_classification(n_samples=300,
                          n_features=2,
                          n_redundant=0,
                          n_informative=2,
                          n_clusters_per_class=1,
                          random_state=42)

model = MLPClassifier(hidden_layer_sizes=(10, 10),
                      activation='relu',
                      max_iter=1000,
                      random_state=42)

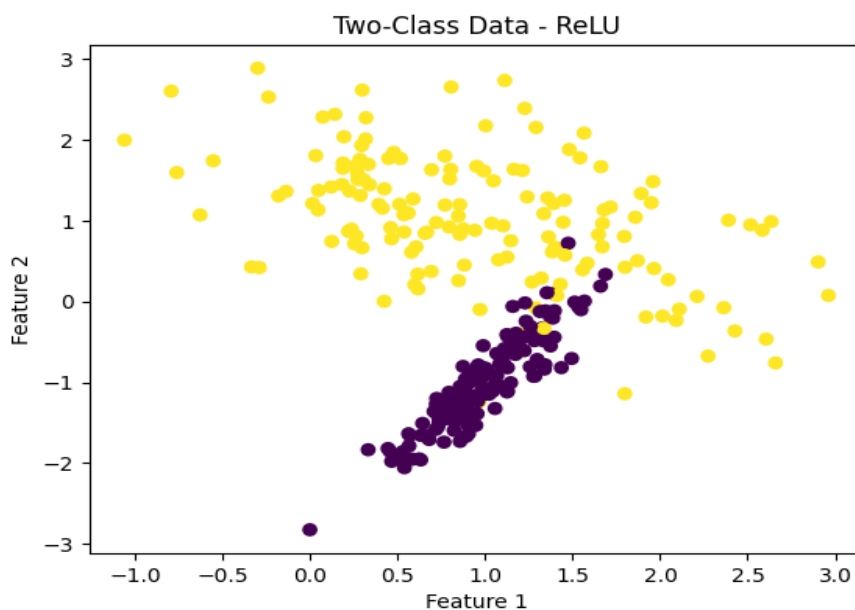
model.fit(X, y)
accuracy = model.score(X, y)
print("Activation Function: ReLU")
print("Number of Classes:", 2)
print("Accuracy:", round(accuracy * 100, 2), "%")
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Two-Class Data - ReLU")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT:

Activation Function: ReLU

Number of Classes: 2

Accuracy: 96.0 %



EXP 23:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPClassifier
np.random.seed(42)
n = 300
theta = np.sqrt(np.random.rand(n)) * 2 * np.pi
r = 2 * theta
X1 = np.c_[r * np.cos(theta), r * np.sin(theta)]
X2 = np.c_[-r * np.cos(theta), -r * np.sin(theta)]
X = np.vstack((X1, X2))
y = np.hstack((np.zeros(n), np.ones(n)))
model = MLPClassifier(hidden_layer_sizes=(20, 20),
                      activation='logistic',
                      max_iter=2000,
                      random_state=42)

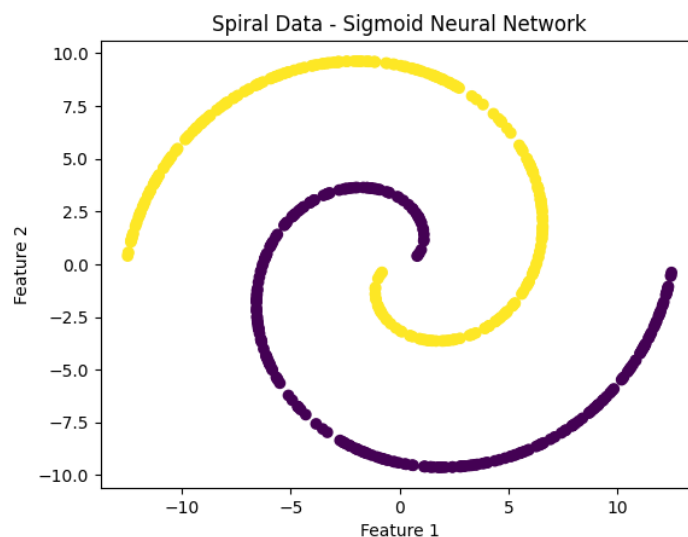
model.fit(X, y)
accuracy = model.score(X, y)
print("Activation Function: Sigmoid")
print("Classes:", 2)
print("Accuracy:", round(accuracy * 100, 2), "%")
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Spiral Data - Sigmoid Neural Network")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT:

Activation Function: Sigmoid

Classes: 2

Accuracy: 98.0 %



EXP 24:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neural_network import MLPClassifier

X, y = make_classification(n_samples=400,
                           n_features=2,
                           n_redundant=0,
                           n_informative=2,
                           n_classes=3,
                           n_clusters_per_class=1,
                           random_state=42)

model = MLPClassifier(hidden_layer_sizes=(15, 15),
                      activation='logistic',
                      max_iter=1500,
                      random_state=42)

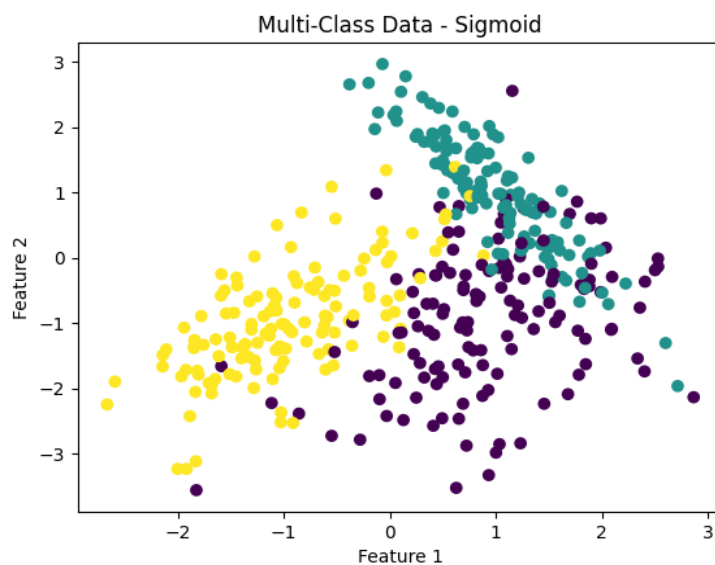
model.fit(X, y)
accuracy = model.score(X, y)
print("Activation Function: Sigmoid")
print("Number of Classes:", 3)
print("Accuracy:", round(accuracy * 100, 2), "%")
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi-Class Data - Sigmoid")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT

Activation Function: Sigmoid

Number of Classes: 3

Accuracy: 85.0 %



EXP 25:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.neural_network import MLPClassifier

X, y = make_circles(n_samples=300,
                    noise=0.08,
                    factor=0.5,
                    random_state=42)

model = MLPClassifier(hidden_layer_sizes=(10, 10),
                    activation='tanh',
                    max_iter=1000,
                    random_state=42)

model.fit(X, y)
accuracy = model.score(X, y)
print("Activation Function: Tanh")
print("Input Samples:", len(X))
print("Accuracy:", round(accuracy * 100, 2), "%")
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Circular Data - Tanh Neural Network")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

OUTPUT

Activation Function: Tanh

Input Samples: 300

Accuracy: 100.0 %

